

Special and common aspects of pdf/dvi/xdvi generators

Ernst Reissner (rei3ner@arcor.de)

2025-11-23

Contents

List of Tables	1
List of Listings	2
1 Introduction	2
2 The options for T_EX Live	2
3 The options for MiKTeX	4
4 Reproducibility and security for PDF documents created	6
4.1 The tool chains	7
4.2 Dates, times and time zones	7
4.2.1 Date/time formats and time zones	7
4.2.2 Date, time and time zones in PDF files created from L ^A T _E X files	9
4.3 Metadata in the PDF trailer and info dictionary	11
4.4 Reproducibility	13
4.5 Suppressing and overwriting meta info in an engine specific way	14
4.6 Security and Stability of Reproducibility	14
4.7 Miscellaneous	15
4.7.1 The command \DocumentMetadata	15
4.7.2 Manipulating the trailer identifier	16
5 To be clarified	16
6 References	16

List of Tables

1	L ^A T _E X engines and the version this document refers to	2
2	Options of T _E X engines in T _E X Live	4
3	Options of T _E X engines in MiKTeX	6
4	Keys of DID and replacement by XMP	12

List of Listings

1	Definition of <code>\printdatetime</code>	10
---	---	----

1 Introduction

This document is created with `lualatex` or that like with output format PDF. The package `tex4ht` is not loaded.

This document is about features the three $\text{\LaTeX}$ engines, `pdf $\text{\LaTeX}$` , `lua $\text{\LaTeX}$` and `x $\text{\LaTeX}$` have in common and discusses also aspects under which they are specific. These programs are just underlying $\text{\TeX}$ engines preloading the $\text{\LaTeX}$ format. The names of the underlying engines just drop the inner syllable “`la`”. For `pdf $\text{\TeX}$` there is a user manual [THHB24], for `lua $\text{\TeX}$` there is a reference manual [HHHS24] and for `x $\text{\TeX}$` a reference guide [RHB24].

The first aspect we cover are the options (among those to display the version). It turns out, that the options are specific for the distribution. Apart from $\text{\TeX}$ Live, there is a second important distribution, MiK $\text{\TeX}$, which should also be treated.

We treat the options for $\text{\TeX}$ Live in Section 2 and those for MiK $\text{\TeX}$ in Section 3.

The second subject is privacy as part of security and reproducible builds of documents, e.g. for tests. The focus is here on PDF files and comprises besides visible data also metadata. The results of our research is collected in Section 4.

2 The options for $\text{\TeX}$ Live

Note that in fact we use a variant of `luatex`, called `lua $\text{\LaTeX}$` .

This document is valid for versions of the underlying $\text{\LaTeX}$ engine as given in Table 1. Moreover, our research refers to a specific distribution, $\text{\TeX}$ Live.

$\text{\LaTeX}$ engine	version
<code>pdf$\text{\LaTeX}$</code>	pdf $\text{\TeX}$ 3.141592653-2.6-1.40.24
<code>x$\text{\LaTeX}$</code>	Xe $\text{\TeX}$ 3.141592653-2.6-0.999994
<code>lua$\text{\LaTeX}$</code>	LuaHB $\text{\TeX}$, Version 1.15.0

Table 1: $\text{\LaTeX}$ engines and the version this document refers to

We start with a synopsis of the options. Table 2 shows options of the $\text{\LaTeX}$ engines under consideration. Note that in contrast to the other $\text{\LaTeX}$ engines, `lua $\text{\LaTeX}$` defines options starting with `--` but it can also process the options if given with a single dash. Conversely, $\text{\LaTeX}$ engines other than `lua $\text{\LaTeX}$` can also deal with options starting with double dash. Options unknown to a $\text{\LaTeX}$ engine never result in an error or even a warning; instead just an info message ‘unrecognized option’ is displayed. This allows to create a configuration which works for all $\text{\LaTeX}$ engines.

In Table 2 column “included”, each $\text{\LaTeX}$ engine is represented by the starting letter of its name, so for each option it is known which $\text{\LaTeX}$ engines know about it and conversely, which options each $\text{\LaTeX}$ engine has.

The table allows furnishing configurations working for all $\text{\LaTeX}$ engines. Some options are common to all $\text{\LaTeX}$ engines

option	included			explanation
(-)-cnf-line=STRING	p	x	1	parse STRING as a configuration file line
--credits	-	-	1	Display credits and exit.
--debug-format	-	-	1	enable format debugging
(-)-draftmode	p	-	1	switch on draft mode (generates no output PDF)
-enc	p	-	-	Enable encTeX extensions such as <code>\mubyte</code>
-etex	p	x	-	enable e-TeX extensions
(-)-[no-]file-line-error	p	x	1	disable/enable file:line:error style messages
--[no-]file-line-error-style	-	-	1	aliases of <code>--[no-]file-line-error</code>
-fmt=FMTNAME	p	x	1	use FMTNAME instead of program name or a <code>%&</code> line ¹
(-)-ini	p	x	1	for dumping formats
-ipc	p	-	-	send DVI output to a socket
-ipc-start	p	-	-	as well as the usual output file
(-)-halt-on-error	p	x	1	as <code>-ipc</code> , and also start the server at the other end
(-)-help	p	x	1	stop processing at the first error
(-)-version	p	x	1	display this help and exit
-8bit	p	x	-	output version information and exit
(-)-interaction=STRING	p	x	1	make all characters printable by default ²
				set interaction mode
				(STRING=batchmode/nonstopmode/ scrollmode/errorstopmode)
(-)-jobname=STRING	p	x	1	set the job name to STRING
(-)-kpathsea-debug=NUMBER	p	x	1	set path searching debugging flags
				according to the bits of NUMBER
--lua=FILE	-	-	1	Load and execute a lua initialization script.
--luaonly	-	-	1	run a lua file, then exit
--luaconly	-	-	1	byte-compile a lua file, then exit
--luahashchars	-	-	1	the bits used by current Lua interpreter for strings hashing
(-)-[no-]mktex=FMT	p	x	1	disable/enable mktexFMT generation ³
-mltex	p	x	-	enable MLTeX extensions such as <code>\charsubdef</code>
-no-pdf	-	x	-	generate XDV (extended DVI) output rather than PDF
--nosocket	-	-	1	Disable the Lua socket library.
(-)-output-comment=STRING	p	x	1	use STRING for DVI file comment instead of date
				(no effect for PDF) ⁴
(-)-output-directory=DIR	p	x	1	use existing DIR as the directory to write files in
(-)-output-format=FORMAT	p	-	1	use FORMAT for job output; FORMAT is ‘dvi’ or ‘pdf’ ⁵
-output-driver=CMD	-	x	-	use CMD as the XDV-to-PDF driver instead of <code>xdvipdfmx</code>
-papersize=STRING	-	x	-	set PDF media size to STRING
-[no-]parse-first-line	p	x	-	disable/enable parsing of first line of input file
(-)-programe=STRING	p	x	1	set program (and fmt) name to STRING ⁶
(-)-recorder	p	x	1	enable filename recorder
--safer	-	-	1	Disable easily exploitable Lua commands.
(-)-[no-]shell-escape	p	x	1	disable/enable <code>\write18SHELL COMMAND</code> ⁷
(-)-shell-restricted	p	x	1	enable restricted <code>\write18</code> ⁸
-src-specials	p	x	-	insert source specials into the DVI file
-src-specials=WHERE	p	x	-	insert source specials in certain places of
				the DVI/XDV ⁹ file.
(-)-synctex=NUMBER	p	x	1	generate SyncTeX data for previewers ¹⁰
-translate-file=TCXNAME	p			use the TCX file TCXNAME ¹¹

¹in fact for `lualatex` the explanation deviates a bit: `--fmt=FORMAT`: load the format file FORMAT

²for `xelatex`: don’t use `^^X` sequences

³(FMT=tex/tfm/pk) for `pdflatex`; else (FMT=tex/tfm)

⁴For `xelatex` it is XDV instead of DVI and the remark (no effect for PDF) is missing

⁵`xelatex` offers option `-no-pdf` instead.

⁶`lualatex` does not mention (and fmt)

⁷For `lualatex` the explanation is disable/enable system commands

⁸For `lualatex` the explanation is restrict system commands to a list of commands given in `texmf.cnf`

⁹DVI for `pdflatex`; XDV for `xelatex`

¹⁰Explanation differs for `lualatex`

¹¹TCX means $\TeX$ character translation

Table 2: Options of T_EX engines in T_EX Live

pdflatex:

Usage: pdftex [OPTION]... [TEXNAME[.tex]] [COMMANDS]
 or: pdftex [OPTION]... \FIRST-LINE
 or: pdftex [OPTION]... &FMT ARGS
 Run pdfTeX on TEXNAME, usually creating TEXNAME.pdf.
 Any remaining COMMANDS are processed as pdfTeX input, after TEXNAME is read.
 If the first line of TEXNAME is %&FMT, and FMT is an existing .fmt file,
 use it. Else use `NAME.fmt', where NAME is the program invocation name,
 most commonly `pdftex'.

Alternatively, if the first non-option argument begins with a backslash,
 interpret all non-option arguments as a line of pdfTeX input.

Alternatively, if the first non-option argument begins with a &, the
 next word is taken as the FMT to read, overriding all else. Any
 remaining arguments are processed as above.

If no arguments or options are specified, prompt for input.

3 The options for MiKTeX

Since at the time of this writing, the author has no MiKTeX at hand, the results for MiKTeX are based on documentation, rather than experimentation. The three engines are a bit different, also in their names.

Well this section is preliminary only. It turned out that Section 2 is valid only for distribution T_EX Live. So in this section we venture to find out the options for the other big distribution, MiKTeX. We shall also investigate whether there are further distributions.

Whereas the description [KB23] seems not to mention the options explicitly, the MiKTeX manual [Sch22] describes each program in Section II, 6, in particular also the L^AT_EX engines. This is the source of the following tables.

The first observation is that, for MiKTeX all options start with two dashes, whereas for T_EX Live this is the case only for luatex. One has to clarify, whether the maven latex plugin under consideration really works for MiKTeX.

option	included			explanation
--alias=name	p	x	1	Pretend to be program name, ... ¹³
--aux-directory=dir	p	x	1	Set dir as the directory to write auxiliary files to.
--buf-size=n	p	x	-	Set the the maximum number of characters ...
--c-style-errors	p	x	1	Change the way, error messages are printed.
--credits	-	-	1	Display credits and exit ¹⁴ .
--disable-8bit-chars	p	x	-	Make only 7-bit characters printable.
--disable/enable-installer	p	x	1	Disable/Enable automatic installation of packages.

¹²Coordinated Universal Time, successor of Greenwich Mean Time (GMT)

¹³Using this option to copying the program file to name and invoking name.

¹⁴The same as for T_EX Live.

<code>--disable-write18</code>	p	x	1	Disable the <code>\write18{command}</code> construct.
<code>--enable-write18</code>	p	x	1	Fully enable the <code>\write18{command}</code> construct ¹⁵ .
<code>--restrict-write18</code>	p	x	1	Partially enable the <code>\write18command</code> construct.
<code>--debug-format</code>	-	-	1	Enable format debugging ¹⁶ .
<code>--[dont-]parse-first-line</code>	p	x	-	[Dont p[P]arse first line of input file under definite conditions ¹⁷
<code>--draftmode</code>	p	-	1	switch on draft mode (generates no output PDF) ¹⁸
<code>--enable-8bit-chars</code>	p	x	-	Make all characters printable.
<code>--enable-encTeX</code>	p	-	-	Enable encTeX extensions such as <code>\mubyte</code> ¹⁹ .
<code>--enable-etex</code>	p	x	-	Enable eTeX extensions.
<code>--enable-installer</code>	p	x	1	Enable automatic installation of packages.
<code>--enable-mltex</code>	p	x	-	Enable MLTeX extensions such as <code>\charsubdef</code> .
<code>--error-line=n</code>	p	x	-	Set the width of context lines on ..error messages.
<code>--extra-mem-bot=n</code>	p	x	-	Set the extra size ... for large data structures ...
<code>--extra-mem-top=n</code>	p	x	-	Set the extra size (in memory words) for chars, tokens,
<code>--font-max=n</code>	p	x	-	Set the maximum internal font number.
<code>--font-mem-size=n</code>	p	x	-	Set the size, in TeX memory words, of the font memory.
<code>--half-error-line=n</code>	p	x	-	Set the width of first lines of contexts in terminal error messages.
<code>--halt-on-error</code>	p	x	1	Quit after the first error.
<code>--hash-extra=n</code>	p	x	-	Set the extra space for the hash table of control sequences ...
<code>--help</code>	p	x	1	Give help and exit ²⁰ ..
<code>--hhhelp</code>	p	x	-	manual page in an HTML Help window ²¹
<code>--include-directory=dir</code>	p	x	1	Add the directory <code>dir</code> to [those] to be searched for input files.
<code>--initialize</code>	p	x	1	Become the INI variant of the program.
<code>--interaction=mode</code>	p	x	1	Set the interaction mode (<code>mode=batchmode/nonstopmode/scrollmode/errorstopmode</code>).
<code>--job-name=name</code>	p	x	1	Set the name of the job (<code>\jobname</code>).
<code>--job-time=file</code>	p	x	-	Set the time-stamp of all output files equal to file's time-stamp.
<code>--lua=FILE</code>	-	-	1	load and execute a lua initialization script ²² .
<code>--luaonly</code>	-	-	1	Start LuaTeX as a Lua interpreter ²³ .
<code>--luaconly</code>	-	-	1	byte-compile a lua file, then exit ²⁴ .
<code>--luahashchars</code>	-	-	1	the bits used by current Lua interpreter for strings hashing
<code>--main-memory=n</code>	p	x	-	Change the total size ... of the main memory array.
<code>--max-in-open=n</code>	p	x	-	Set the maximum number of input files ...
<code>--max-print-line=n</code>	p	x	-	Set the width of longest text lines output.
<code>--max-strings=n</code>	p	x	-	Set the maximum number of strings.
<code>--[no-]mktex=fmt</code>	-	-	1	Enable/Disable <code>fnt</code> generation, where <code>fnt</code> must be either <code>tex</code> or <code>tfm</code> .
<code>--nest-size=n</code>	p	x	-	Set the maximum number of semantic levels simultaneously active.
<code>--no-c-style-errors</code>	p	x	1	Don't change the way, error messages are printed.
<code>--no-pdf</code>	-	x	-	generate XDV (extended DVI) output rather than PDF
<code>--nosocket</code>	-	-	1	Disable the Lua socket library.
<code>--output-comment=string</code>	-	-	1	Use <code>string</code> for DVI file comment instead of date.
<code>--output-directory=dir</code>	p	x	1	Write output files in <code>dir</code> ²⁵ .
<code>--output-driver=CMD</code>	-	x	-	use <code>CMD</code> as the XDV-to-PDF driver instead of <code>xdvipdfmx</code>

¹⁵Corresponds roughly to `(-)-shell-escape` in TeX Live.

¹⁶The same as for TeX Live.

¹⁷Similar for TeX Live. Note that there is also a converse option.

¹⁸Some differences in formulation between the L^ATeX engines and also between distributions

¹⁹Corresponds with `-enc` in TeX Live.

²⁰The same as for TeX Live.

²¹This option is only available on Windows systems.

²²The same as for TeX Live.

²³Could be the same as for TeX Live.

²⁴Could be the same as for TeX Live.

²⁵Similar as for TeX Live.

<code>--output-format=format</code>	p	-	1	Use format for job output (one of: dvi, pdf) ²⁶ .
<code>--papersize=STRING</code>	-	x	-	set PDF media size to STRING
<code>--param-size=n</code>	p	x	-	Set the the maximum number of simultaneous macro parameters.
<code>--pool-free=n</code>	p	x	-	Set the minimum pool space left after loading the format.
<code>--pool-size=n</code>	p	x	-	Set the maximum number of characters in strings, ...
<code>--quiet</code>	p	x	-	Suppress all output, except errors.
<code>--record-package-usages=file</code>	p	x	-	Record all package usages and write them into file.
<code>--recorder</code>	p	x	1	Enable the file name recorder ²⁷ .
<code>--safer</code>	-	-	1	Disable easily exploitable Lua commands ²⁸ .
<code>--save-size=n</code>	p	x	-	Set the the amount of space for saving values outside of current group.
<code>--src-specials</code>	p	x	-	Embed source file information in the DVI file ²⁹ .
<code>--stack-size=n</code>	p	x	-	Set the maximum number of simultaneous input sources.
<code>--string-vacancies=n</code>	p	x	-	Set the minimum number of characters ...
<code>--synctex=n</code>	p	x	1	Generate SyncTeX data for previewers ³⁰
<code>--tcx=tcxname</code>	p	-	-	Use the tcxname translation table ...
<code>--time-statistics</code>	p	x	-	Show processing time statistics.
<code>--trace[=tracestreams]</code>	p	x	-	Enable trace messages.
<code>--trie-size=n</code>	p	x	-	Set the amount of space for hyphenation patterns.
<code>--undump=name</code>	p	x	1	Use name as the name of the format to be used, ...
<code>--utc</code>	-	-	1	Init time to UTC ³¹ .
<code>--version</code>	p	x	1	Show version information and exit ³² .

Table 3: Options of T_EX engines in MiKTeX

Strange, there are `--enable-etex` and `--enable-mttx` but no way to disable. Maybe disable is the default.

```
miktex-pdfTeX [option...] [[file] | [\command...]]
```

```
miktex-luatex [option...] [[command...] | [file]]
```

The following options are ignored:

`--8bit`, `--etex`, `--parse-first-line`, `--no-parse-first-line`
These are always on.

`--default-translate-file=tcxname`, `--translate-file=tcxname`
These are always off.

```
miktex-xetex [option...] [[file] | [\command...]]
```

4 Reproducibility and security for PDF documents created

Of course reproducibility and security are general subjects not tied to a specific format like PDF, but as a first step we undertake a discussion specific for PDF files. This shall be extended step by step.

²⁶`pdfLatex` and `luaLatex` differ a bit in text. Seems similar to T_EX Live.

²⁷The same as in T_EX Live.

²⁸The same as for T_EX Live.

²⁹Similar as in T_EX Live.

³⁰Explanation with more detail than for T_EX Live.

³¹The same as for T_EX Live.

³²The same as for T_EX Live.

Reproducible builds are important for tests and for global cooperation. For PDF files besides visible data also so-called metadata must be reproduced. Security is mainly privacy here. Also, besides visible data also metadata shall not expose private data.

To display metadata, we use `exiftool` and `pdftools`.

4.1 The tool chains

Although `xelatex` always produces a XDV file as an intermediate step, when creating a PDF file this is eliminated. In contrast, with the option `-no-pdf` one can eliminate creation of the PDF file and the XDV is not erased. For conversion, of the XDV file to PDF, the option `-output-driver=CMD` is used which defaults to the command `xdvipdfmx`. Besides direct creation of a PDF file, we consider creation via XDV file using `xdvipdfmx`. The XDV format is an extension and in fact a variant of the DVI format.

For the other engines in contrast, the option `-output-format=dvi/pdf` determines the output format which is PDF by default and there is no intermediate format for PDF. When creating DVI files instead, these files can be converted into PDF by explicitly invoking something like `dvipdfmx`, `dvipdfmx` or `xdvipdfmx`. In my current distribution $\text{\TeX}$ Live, the programs `dvipdfm`, `dvipdfmx` and `xdvipdfmx` are all binary identical.

Nevertheless, they turn out to yield different results. One reason found below is, that the name with which the program is invoked goes into the result. It is likely that this is the only reason.

As a consequence of the workflow of `xelatex`, `\ifpdf` provided by package `iftex` always enters the `\else` branch for `xelatex`.

Although this section focuses on PDF format, of course the intermediate format DVI/XDV must be considered by need. The visual appearance is roughly the same, so visual reproducibility of DVI/XDV is almost equivalent with visual reproducibility of PDF, but for metadata one has to distinguish cases where the $\text{\LaTeX}$ engine already creates metadata and writes it into `\special` commands in a DVI/XDV file, whereas the PDF creator just includes this data from DVI/XDV from cases where it is the PDF creator which creates metadata, like creation time.

4.2 Dates, times and time zones

Clearly, creation times affect reproducibility quite directly, because usually the date is given in the head page of a document, but it goes also into metadata both directly and indirectly.

Before discussing aspects of time in the context of PDF creation from $\text{\LaTeX}$ in Section 4.2.2, Section 4.2.1 provides the general foundation.

4.2.1 Date/time formats and time zones

This section introduces time zones, date/time formats and conversions between them.

We start with *epoch time* which is the most important date/time format, although not well human-readable. It is the number of seconds from start of computer time history, which is agreed to be

1970-01-01T00:00:00Z

i.e. the point in time, where in timezone UTC (represented by the Z, zero deviation from UTC, also called Zulu-time) it was start of year 1970, i.e. date 1970-01-01 and midnight, time 00:00:00. It is called epoch time, although nothing special happened like Christs birth. Zero and negative time is allowed. Note, that this does not depend on a time zone, but as human-readable time

formats depend on the time zone, conversion from and to epoch time must take time zone into account.

In fact, the above date is an example of a representation given by [ISO19]. More general, the timezone is either Z representing UTC or some deviation from UTC for example given with two digits for hours and two digits for minutes. The same date and time as above in time zone two hours earlier (more east) than UTC writes

```
1970-01-01T02:00:00+0200
```

In POSIX operating systems, among the standard environment variables is also TZ specifying the time zone as described in https://www.gnu.org/software/libc/manual/html_node/Standard-Environment.html. We only use the forms based on UTC like so:

```
TZ=UTC, TZ=UTC-02
```

CAUTION: The number is understood as to be added to given time in the current time zone to get UTC, so east of UTC has negative numbers. This is the sign opposite to the representation given by [ISO19] which is the offset to be added to UTC to get the time in the given time zone.

In Unix and similar operating systems, conversion can be done with **date**. If a time is given with option **-d**, this is converted into the format given by the format specifier which starts with **+**. If no time is given, the current time is assumed.

date yields the current date and time with timezone in current locale representation. The locale is given by the environment variable **LANG**.

date +%FT%T%z yields the current date and time in form given by ISO 8601 with date given by **%F** followed by literal T followed by time **%T** and finally the time zone given by **%z**.

date -d 'Apr 29 2024' +%s converts human-readable date into epoch time specified by format **%s** (seconds since 1970-01-01, i.e. epoch time). **Caution:** Many formats are recognized, but time zones are *silently ignored*. For example,

```
date -d '2024-06-20T12:03:00+03' +%s
```

silently ignores the timezone +03 and uses the current one which is UTC+02, because me the author is located in Germany. This can be seen, because the result does not depend on the time zone; you can use +02 or Z signifying UTC or drop: the result is always 1718877780.

To set the time zone, the variable TZ must be used like so:

```
TZ=UTC-02 date -d '2024-06-20T12:03:00' +%s
```

Note the sign difference between ISO 8601 representation and TZ. Since UTC+02 is the timezone of the author, the result is still 1718877780, but if using TZ=UTC-03, which is one hour east, it is one hour, i.e. 3600 seconds less which results in epoch time 1718874180.

date -d '@1718877780' +%FT%T%z converts from epoch time (signified by leading @) into local time zone resulting in

```
2024-06-20T12:03:00+0200
```


Since the time zone of the author is currently `GTM+02`, or with minutes `GTM+0200` this is what the resulting time string ends with.

`TZ=UTC-03 date -d '@1718877780' +%FT%T%z` converts from epoch time (signified by leading `@`) into time zone given by `TZ` resulting in

`2024-06-20T13:03:00+0300`

Compared with the result above, the time which is one hour later and this is reflected by the according time zone. Note again the sign contrary to the value of `TZ`.

Finally, [ISO20], Section 7.9.4 describes the encoding of date/time with timezone in metadata of PDF files. It is quite similar to the format given by ISO 8601–1. Transformation is by dropping separators `--` and `:`, prefixing `D:` and inserting `'` separating hours and minutes in time zone. The above date/time writes as follows:

`D:20240620130300+03'00`

4.2.2 Date, time and time zones in PDF files created from `LATEX` files

When creating documents from `LATEX`, date and time information go into at least in the following aspects: in

- `LATEX` macros displaying time and date in the document like the built-in `\today`, `\date` and `\time` accessible with prefixed `\the`, and also `\DTMnow` provided by package `datetime2`
- metadata of PDF documents like `CreationDate` and `ModDate`
- file properties like creation time and modification time.

There are several environment variables affecting treatment of these aspects. First we analyze behavior without them.

Let us start with the last aspect: For Linux, the command `stat` yields both creation time and modification time of a file `xxx.pdf`:

`stat -c +%w xxx.pdf` is the time of file birth, i.e. creation time

`stat -c +%y xxx.pdf` is the modification time. This is the same as given by `ls -l`.

Both times comprise the date are endowed with time zone information and are given in human-readable form, although not precisely conform with ISO 8601–1. The command `stat` assumes the local time zone by default, but is sensitive to `TZ`:

`TZ=UTC stat -c +%w xxx.pdf`

displays the result in UTC time zone. The according pieces of information in epoch time are accessed with option in capital letters, i.e. `+%W` and `+%Y`, respectively.

As one would expect, the modification time is the time when the engine finished writing. The time of file birth is really when the file comes into existence, e.g. for the first time or after having been deleted.

The first surprise with metadata `CreationDate` and `ModDate` is, that in fact it is not only date but comprises time and timezone as well, much the same as the according file properties. The form in which they are given in the PDF files is described in Section 4.2.1. This is essentially accessible via

```

\ExplSyntaxOn%

\NewExpandableDocumentCommand{\printtime}{}
{
  \int_compare:nT { \c_sys_hour_int < 10 } { 0 }
  \int_eval:n { \c_sys_hour_int }
  :% chkTeX 26
  \int_compare:nT { \c_sys_minute_int < 10 } { 0 }
  \int_eval:n { \c_sys_minute_int }
}

\ExplSyntaxOff%

```

Listing 1: Definition of `\printdatetime`

```
pdfinfo -rawdates xxx.pdf
```

The data is considered raw because it is displayed as is, i.e. as stored in the PDF file. It is close to ISO 8824–1. Dates are also accessible in format specified in ISO 8601–1 via

```
pdfinfo -isodates xxx.pdf
```

The second surprise is, that the creation date and modification date given by metadata do not coincide with the according file properties.

First, `xelatex` is special in that by default, it provides `CreationDate` only, whereas `lualatex` and `pdflatex` provide both. The command `\DocumentMetadata` forces `xelatex` to display both times. This is not really important, because the two times seem to coincide if both are given. The idea behind creation time is, that the `LATEX` engine creates a new file even when overwriting an old one. The time when starting to overwrite seems to be the `CreationDate`. Why the `ModDate` is the same and not the later time when writing is finished, well...the reader may find an explanation him/herself. An idea is that this is related to incremental updates described in [ISO20], Section 7.5.6: Instead of creating a PDF anew if a modification occurs, it is advised to just append a description of the difference. This update does not affect creation date but only modification date. `LATEX` engines do not support incremental updates and so compilation creates just a new PDF document.

Both, `xelatex` and `pdflatex` represent `CreationDate` and `ModDate` with the local time zone which can be overwritten by `TZ`, and so does `lualatex` by default. In addition, `lualatex` has an option `--utc` both for `TEX Live` and for `MiKTeX` described in Sections 2 and 3 which forces metadata given in UTC zone, overwriting internal timezone but also the value of `TZ` if provided.

Finally, we have to consider the visible date and time. These are typically displayed by commands like `\date` or `\time`. We prefer `\Today` provided by package `datetime2` with option `style=iso` which displays the date in the form `yyy-mm-dd`. This can be used also in the metadata. Note that it is not secure to use `\today` because packages overwrite this. For experiments, we may use `\printdatetime` defined in Listing 1 which is based on `\time`.

Time and date in the visible document, e.g. by `\DTMnow` behave exactly as the date and time given by the metadata, except that `xelatex` does not display seconds in visible document and does not make the time zone visible, although metadata show that both pieces of information are available.

Note that as `lualatex` represents metadata in UTC time if invoked with option `--utc`, the same is true for visible date and time. The time zone is made explicit. Note that the variable `TZ` is treated as for metadata: Goes into representation, except for `lualatex` invoked with option `--utc`.

The environment variable `SOURCE_DATE_EPOCH` defined in <https://reproducible-builds.org/docs/source-date-epoch/>, is set with epoch time. It was introduced to obtain reproducible builds in a general context can also be used in $\text{\TeX}$ systems. Although `SOURCE_DATE_EPOCH` defines the build time in a sense, it is specified, that the given epoch time is exactly what is written into the metadata. It affects each $\text{\LaTeX}$ engine but in slightly different way.

For `pdflatex`, the main description is in [THHB24], Chapter 2. An important detail is, that the time zone written into metadata is UTC. Even if the timezone is given explicitly by `TZ`, it is ignored. The time given by `SOURCE_DATE_EPOCH` affects metadata but also `\DTMnow`. To affect also primitives like `\time`, one has to add `FORCE_SOURCE_DATE=1` in addition. Essentially, `xelatex` behaves the same as described in `xelatex`: [RHB24], Sections 8.3 and 8.5.

In contrast to this, as described in [HHHS24], Section 4.4, `lualatex` does honor the environment variable `SOURCE_DATE_EPOCH` but not `FORCE_SOURCE_DATE`. Instead, it acts as if `FORCE_SOURCE_DATE=1` is set implicitly: Besides metadata, also visible date representations honor `SOURCE_DATE_EPOCH`, be it `\time` or `\DTMnow`. In contrast to `pdflatex` and `xelatex`, which automatically display epoch time given by `SOURCE_DATE_EPOCH` in UTC, `lualatex` does so only if invoked with option `--utc`. Else `lualatex` honors the current time zone which may also be specified by `TZ`.

At time of this writing there seems to be a bug in `lualatex` leading to the wrong time zone when the time is start of a full hour.

Last thing: $\text{\LaTeX}$ engines creating DVI/XDV and going on with `xdvipdvmx`. Of course, visible data are as with direct compilation and also file times are straightforward. To get these data right, $\text{\LaTeX}$ engine must be invoked with the system variables as for direct creation of PDF files.

The only issue are meta-data. Here also they seem to be stored in the intermediate dvi/xdv files, and just passed to PDF by `xdvipdvmx`, except date and time information. These are added by `xdvipdfmx`.

So, `SOURCE_DATE_EPOCH` and `TZ` but not `FORCE_SOURCE_DATE` must be considered when invoking `xdvipdfmx` and that like. If none of these are given, the current time with local time zone are used. If `SOURCE_DATE_EPOCH` is not given, then `TZ` is honored, but if `SOURCE_DATE_EPOCH` is given, then creation time is in UTC time zone, ignoring `TZ`. So `xdvipdfmx` behaves very much like `xelatex` in direct compilation of PDF, which is clear because it is by defaults `xelatex`'s PDF backend. This behavior of `xdvipdfmx` coins time metadata even if used in conjunction with `lualatex`, which is some argument to configure direction computation of PDF files with `lualatex` in a way that it fits the other engines.

4.3 Metadata in the PDF trailer and info dictionary

The root for metadata in PDF file is the *trailer dictionary*. As described in [ISO20], Section 7.5.5, it contains references to several other dictionaries, the size and the *trailer identifier* with key `ID`. Among the references is that with key `Info` to the *document information dictionary* (DID) described in [ISO20], Section 14.3.3. The most important entries are `CreationDate` and `ModDate`; the rest is deprecated with PDF version 2.0. Still these entries are relevant for PDF versions 1.x. For all entries in the document information dictionary including `CreationDate` and `ModDate` there are replacements in the metadata streams described in [ISO20], Section 14.3.2. To be more precise it is the XMP metadata, which is an XML format, described in [ISO12]. Frequently, the replacements allow a set of locales. The $\text{\LaTeX}$ engine writes this if in the header of the TEX file

`\DocumentMetadata{...}`

is specified, even if the argument list is empty, because this signifies default `xmp=true`. The result can be read through

`pdftools -meta xxx.pdf`

The keys of the document information dictionary (DID) and their replacements as XMP are given in Table 4. Note that the keys for XMP have a prefixed namespace separated by `:`. The table does not list all XMP keys specified in [ISO12], but only those which are observed as created by a $\text{\LaTeX}$ engine. Observe that the XMP keys with namespace `pdf` are not specified in [ISO12].

DID	XMP	remark
Title	dc:title	same
Author	dc:creator	same
Subject	dc:description	
Keywords	dc:subject	not pdf:Keywords as specified
—	dc:type	
—	dc:language	
—	dc:date	duplicate of CreationDate?
—	dc:format	application/pdf
—	dc:source	the $\text{\LaTeX}$ main file
Creator	xmp:CreatorTool	should be creator of tex source...
Producer	pdf:Producer	
—	pdf:PDFVersion	
CreationDate	xmp:CreateDate	
ModDate	xmp:ModifyDate	same as CreationDate?
—	xmp:MetadataDate	same as CreationDate?
Trapped	pdf:Trapped	
—	xmpMM:DocumentID	
—	xmpMM:InstanceID	

Table 4: Keys of DID and replacement by XMP

So far, the only piece of data degrading security is `dc:source`. One may think of eliminating it using package `hyperxmp`, described in [Pak24]. This is deferred because it collides with `\DocumentMetadata`. The package can even not be loaded if `\DocumentMetadata` as shown in [Pak24], Section 3.1. For this task consider post-processing combining `exiftool` unlinking metainformation and `qpdf` removing by linearizing. At least, `pdftools` makes meta info visible, questionable whether completely.

Besides a reference to the document information dictionary, the trailer dictionary also contains the trailer identifier described in [ISO20], Section 7.5.5. It indicates that not writing the trailer identifier or corrupting it is no good idea, e.g. because it is needed for encryption or for uniquely identifying the document.

Details are given in [ISO20], Section 14.4. First, the identifier consists of two hashes, the first of which depends on the newly created but unmodified document. The notion behind is that of incremental updates described in [ISO20], Section 7.5.6. The first hash shall depend on the content of the file at creation time and on `CreateDate`. The second hash in contrast shall depend on content of the file after last incremental update and on time of last update, i.e. on `ModDate`.

In the $\text{\LaTeX}$ world, incremental update is not supported so that `CreateDate` and `ModDate` coincide and so do the contents. As a consequence, the two hashes of the trailer identifier coincide as well. The last section of [ISO20], Section 14.4 seems to indicate, that the location of the PDF file may go into the trailer ID, which would make it less reproducible.

4.4 Reproducibility

The first observation is, that invocation of `xelatex latexEngines` produces different PDF output for each run. Likewise, `xelatex -no-pdf latexEngines` produces different XDV output for each run. As turns out later, this is because the creation time goes into the result.

Thus, it is plausible that, to obtain reproducibility, we invoke the engine as

```
SOURCE_DATE_EPOCH=0 FORCE_SOURCE_DATE=1 xelatex          latexEngines
SOURCE_DATE_EPOCH=0 FORCE_SOURCE_DATE=1 xelatex -no-pdf latexEngines
```

The second result is, that creating the PDF file and the XDV file that way is reproducible.

It turns out, that this compression setting does not refer to the XDV file, which seems always compressed, but solely to the PDF file. The PDF files differ mainly in the time stamp but also in some hashes which may depend on the time stamp.

Now let us experiment with `xdvipdfmx`. Even if we start with a reproducible XDV file, the PDF file created by `xdvipdfmx` changes with each invocation. This changes when also `xdvipdfmx` is invoked with fixed time.

```
SOURCE_DATE_EPOCH=0 FORCE_SOURCE_DATE=1 xdvipdfmx latexEngines
```

As mentioned above, in the distribution $\text{\TeX}$ Live current at time of this writing, the programs `dvipdfm`, `dvipdfmx` and `xdvipdfmx` are all binary identical. Nevertheless, they seem to lead to different output. Possibly, the invocation name goes into the result. To find out, we do not allow compression. It turns out that the names go into the result as the producer.

Using the package `hyperref`, one can overwrite a lot of metadata. Details are found in the manual [RO22], Section 5.10. In particular, the producer can be set `unknown`. As a result, the trailer identifier is the only remaining difference. Seemingly, the producer goes into this whether displayed or not. The trailer identifier cannot be overwritten by `hyperref`, but only in a way specific for `xelatex`:

```
\special{pdf:trailerid [
  <00112233445566778899aabbccddeeff>
  <00112233445566778899aabbccddeeff>
]}
```

makes even the XDV to PDF converter transparent.

In [RO22], Section 5.10, also the creator is found, which is `LaTeX with hyperref` independent of the $\text{\LaTeX}$ engine. This shall be overwritten if there are security concerns.

As long as the tool chain and settings remain constant, invocation of $\text{\LaTeX}$ engine `xelatex` and backend XDV to PDF converter specifying `SOURCE_DATE_EPOCH=0 FORCE_SOURCE_DATE=1` suffices to guarantee reproducibility. This setting refers to start of computer time history, although nothing special happened like Christs birth, creation date `1970--01--01 00:00:00Z` and if the file exist, this is the modification date. From the point of view of reproducibility, there is ok, but it is not the truth. Thus, it makes sense to overwrite this with the string `unknown`. If the file is overwritten, the same considerations apply to the modification date. Both can be overwritten with package `hyperref`.

One question remains: how does `hyperref` manipulate the metadata and in a second step, can we do this directly without using `hyperref`.

Now let us switch to the other two $\text{\LaTeX}$ engines. Both write the banner information indicating above all the type of engine and the version. As checked by switching compression of, `xelatex` does not write any banner information. Again, as long as the engine does not change nor changes its version or its distribution, the banner does not corrupt reproducibility. On the other hand, removing it would stabilize and generalize reproducibility somewhat: stabilize because the banner contains version information and thus breaks reproducibility at version change, and it breaks reproducibility when changing the engine of course. Also, privacy or security is an argument in favor of eliminating the banner. The package `hyperref` offers no way to change the banner; this can be done only in a machine specific way. The details are described in Section 4.6 below.

The same is true for the trailer identifier. Strictly speaking it need not be suppressed for reproducibility, but to make the result independent of the DVI to PDF converter as is explained in the context of `xelatex` above.

Creating DVI files with `SOURCE_DATE_EPOCH=0 FORCE_SOURCE_DATE=1` yields reproducible results. As expected, these settings are also necessary for translating DVI into PDF.

4.5 Suppressing and overwriting meta info in an engine specific way

For `lualatex` [HHHS24], Section 3.2.2 describes the variable `suppressoptionalinfo` which specifies bitwise which pieces of metadata to be suppressed. Among the PTEX entries, `PTEX.FullBanner` is unique as it refers to the top level document: it is the first line of output in the log file. For sake of privacy this shall be suppressed. The other PTEX entries, `PTEX.FileName`, `PTEX.PageNumber` and `PTEX.InfoDict` refer to embedded PDF files. The `InfoDict` has entries `Producer`, `Creator`, `CreationDate`, `ModDate` and `Trapped` and must be suppressed as well.

Also, `Creator` and `Producer` shall be suppressed. Note that suppression of `Creator` is impossible when using package `hyperref`, because it writes `LaTeX with hyperref` setting itself as creator. To be safe that this piece of information is not exposed, this meta info must be overwritten later. Metadata `Trapped` is suppressed, although there are reasons both for suppression and for leaving as is.

As described elsewhere, neither `CreationDate`, `ModDate` or `ID` shall be suppressed.

As described in [THHB24], Section 4.2.3 and 4.2.4, `pdflatex` can suppress only PTEX data which must be done, and `CreationDate`, `ModDate` which must not be suppressed. Finally, `xelatex` cannot suppress any metadata.

Meta info may be suppressed but also overwritten. For `lualatex` [HHHS24], Section 3.2.2 shows how to overwrite the trailer ID. In contrast, `pdflatex` has many options to overwrite metadata collected in [THHB24], Section 4.2, among those also the trailer ID. Neither of these are needed. Finally, there is no official documentation on `xelatex` for overwriting metadata but internet sources show a way to overwrite the trailer ID:

```
\special{pdf:trailerid [
  <00112233445566778899aabbccddeeff>
  <00112233445566778899aabbccddeeff>
]}
```

4.6 Security and Stability of Reproducibility

Security is here privacy. Hiding information makes attacks more difficult. Stability of reproducibility consists in stability as regards new versions of the same tools in the tool chain and the

aspect of change of a tool or some other aspect of environment.

One aspect is the time zone. As indicated in Section 4.2.2, reproducible PDF files must have `CreateDate` and `ModDate` both in UTC time zone. This supports both privacy, as the true time zone of the build facility is not accessible and stability of reproducibility as a change of location of the build facility or change of summer/winter time may not corrupt reproducibility. This feature supports international cooperation.

From Section 4.4 come the recommendation to set the following pieces of information to unknown., This is done with `hyperref` package ****

Creator This is uniformly `LaTeX with hyperref` as long as `hyperref` is loaded, except for `beamer` class (which loads `hyperref` implicitly) for which it is `LaTeX with Beamer class`. If `hyperref` is not loaded, the creator is `TeX` except for `xelatex` which shows engine and creation date. Thus, it is advisable in general for security but without `hyperref` for sake of stability of reproducibility.

Producer This is `xdvipdfmx` with version for creating DVIs and in general for `xelatex`. For creating PDF with `pdflatex` or with `lualatex`, it is something like `pdfTeX-1.40.25` or `LuaTeX-1.18.0`. This shall be hidden for sake of security and stability of reproducibility.

PTEX.Fullbanner is not written by `xelatex`, but for both `lualatex` and `pdflatex`. It can be suppressed in an engine specific way, but not through `hyperref`. The banner exposes tools, versions and distributions. Thus, it shall not be exposed for sake of security and stability of reproducibility.

All these pieces of information and a bit more are suppressed by including `headerSuppressMetaPDF.tex`. Observe that to that end the package `hyperref` is used whenever possible because this technique is not specific for the `LaTeX` engine. Only banner and trailer identifier are suppressed in an engine specific way.

Besides code, `headerSuppressMetaPDF.tex` provides additional info in the comments, but for details which are specific for the individual engines see [HHHS24], Section 14.1.8 for `lualatex`, and [THHB24], Section 4.2 for `pdflatex`. Seemingly, `xelatex` is quite different from the other engines and tends to write fewer pieces of information. It uses an external XDV to PDF converter, `xdvipdfmx` by default. By placing `\special` commands in the TEX file, the user can pass information to the XDV to PDF converter also controlling meta info to some extent. Some details are given in the manual for `xdvipdfmx`, [Tea20], Section 4.1.1.

4.7 Miscellaneous

4.7.1 The command `\DocumentMetadata`

One of the subjects is the command `\DocumentMetadata` described in [MF23], Section 2. We used it as

```
\DocumentMetadata{uncompress}
```

to avoid compression of the created PDF file, mainly for debugging, but it is significant even with empty argument list, e.g. it forces `xelatex` displaying `ModDate` in addition to `CreationDate`. Another effect, which can be considered a bug is, that overwriting of meta info by command `\hypersetup` offered by package `hyperref` does not work anymore. This applies e.g. to dates but also to creator and producer. This bug does not corrupt reproducibility but security.

Another effect comes into the game because `\DocumentMetadata` has default value `xmp=true` which means that XMP metadata described in [ISO12] is written. This is an XML format and can be retrieved through

```
pdfinfo -meta xxx.pdf
```

Seemingly, this does not corrupt reproducibility but may be considered to degrade security. Partially, it is a surrogate for the document information dictionary as described in Section 4.3.

Readers who refuse using `\DocumentMetadata` and use `xelatex` may take refuge to switching off compression from within the document via

```
\special{xdvipdfmx:config z 0}
```

Equivalently one can use the command line

```
xelatex -shell-escape -output-driver="xdvipdfmx -z 0" <filename>.tex
```

4.7.2 Manipulating the trailer identifier

5 To be clarified

Note that `xelatex` works as the other engines for PDF, whereas it has XDV instead of DVI as alternative format. Thus, the format DVI is ignored. See the manual.

6 References

- [HHHS24] H. Hagen, H. Henkel, T. Hoekwater, and L. Scarso. *LuaTEX Reference Manual*, 2 2024. Refers to version 1.18. A copy is within the documentation of this software.
- [ISO12] International Organization for Standardization (ISO). *Extensible metadata platform (XMP) specification Part 1: Data model, serialization and core properties*, 2012. ISO 16684-1, withdrawn; superseded 2019.
- [ISO19] International Organization for Standardization (ISO). *Date and time — Representations for information interchange Part 1: Basic rules*, 2. 2019. ISO 8601-1.
- [ISO20] International Organization for Standardization (ISO). *Document management – Portable document format – Part 2: PDF 2.0*, 2 edition, 12. 2020. ISO 32000-2:2020.
- [KB23] editor K. Berry. *The TeX Live Guide—2023*, 2 2023. A copy is within the documentation of this software.
- [MF23] F. Mittelbach and U. Fischer. *The documentmetadata-support code*, 3 2023. A copy is within the documentation of this software, in fact two documents, `documentmetadata-support-doc.pdf` and `documentmetadata-support-code.pdf` which also comprises the implementation.
- [Pak24] Scott Pakin. *The hyperxmp package*, 3 2024.
- [RHB24] W. Robertson, K. Hosny, and K. Berry. *The XETEX reference guide*, 3 2024. Refers to version 0.999996. A copy is within the documentation of this software.
- [RO22] Sebastian Rahtz and Heiko Oberdiek. *Hypertext marks in L^AT_EX: a manual for hyperref*, 2 2022.
- [Sch22] C. Schenk. MiKTeX Manual. <https://docs.miktex.org/manual/>, 2022.
- [Tea20] The Dvipdfmx Project Team. *Dvipdfmx User’s Manual*, 6 2020. Version 0.12.4b.

- [THHB24] Han The Thanh, H. Hagen, H. Henkel, and K. Berry. *The pdfTEX user manual*, 2024. For pdftex 1.40.26. A copy is within the documentation of this software.